AWS for Java Backend Developers

A topic-wise reference guide for engineers with 3–5 years of Java backend experience

Contents

1. Compute Services
2. Storage
3. Databases
4. Networking
5. Messaging & Integration
6. Security & Identity
7. API Layer
8. CI/CD & Infrastructure as Code
9. Observability
10. Java-Specific Tooling on AWS

1. Compute Services — Where Your Code Runs

1.1 EC2 (Elastic Compute Cloud)

A virtual machine you rent by the hour. You SSH in, install a JVM, and run your Spring Boot jar — just like a physical server, except AWS owns the hardware.

- **Instance types** — families like `t3` (burstable, cheap, general use), `m5` (balanced), `c5` (compute-optimized), `r5` (memory-optimized). A memory-heavy JVM app usually wants `m5` / `r5`.
- **AMI (Amazon Machine Image)** — the OS + pre-installed software snapshot used to launch an instance.
- **Security Groups** — a virtual firewall attached to the instance; controls which ports/IPs can reach it.
- **Key Pairs** — SSH key used to log in instead of a password.
- **Elastic IP** — a static public IP you can attach/detach from instances.
- **Auto Scaling Groups (ASG)** — automatically add/remove EC2 instances based on load (CPU, request count).
- **Pricing models** — On-Demand (pay per hour, no commitment), Reserved (cheaper, 1–3 year commitment), Spot (cheapest, but AWS can reclaim the instance anytime — good for batch jobs, bad for your primary API server).

> **Why it matters:** Even if your team runs everything on ECS/Lambda today, EC2 concepts (security groups, subnets, AMIs) underpin every other compute service — you can't reason about networking issues without them.

1.2 Elastic Beanstalk

Upload your jar/war; AWS provisions the EC2 instances, load balancer, and auto-scaling for you. A managed shortcut when you don't want to hand-build infrastructure.

- You configure via a simple console/CLI, AWS handles the underlying EC2 fleet.
- Good for smaller apps or quick proof-of-concepts; less flexible than hand-rolled infrastructure for complex needs.

1.3 ECS (Elastic Container Service)

Runs Docker containers. If your Spring Boot app is containerized, this is likely where it runs in production.

- **Task Definition** — a blueprint (like a docker-compose entry) describing the container image, CPU/memory, ports, environment variables.
- **Service** — keeps a desired number of task instances running, integrates with a load balancer, restarts failed tasks.
- **Cluster** — the logical grouping of resources running your tasks.

- **Launch types:**
  - **Fargate** — serverless; you don't manage any EC2 instances, AWS runs your container directly. Simpler, slightly pricier.
  - **EC2 launch type** — you manage the underlying EC2 instances that host containers. Cheaper at scale, more operational overhead.

1.4 EKS (Elastic Kubernetes Service)

Managed Kubernetes. Same pods/deployments/services concepts as any k8s cluster — AWS just manages the control plane. Relevant if your org has standardized on Kubernetes rather than native ECS.

1.5 Lambda

Run a function without provisioning any server. AWS triggers your code in response to an event and only charges for execution time.

- In Java, you implement `RequestHandler<I,O>` as your entry point.
- **Cold start** — the delay when a Lambda spins up a fresh execution environment; Java is notoriously slower here than Node/Python due to JVM startup. **SnapStart** mitigates this by snapshotting an initialized JVM.
- **Triggers** — S3 upload, SQS message, API Gateway request, CloudWatch schedule, etc.
- **Layers** — shared code/dependencies packaged separately from your function code.
- **Timeout & memory** — you configure both; CPU scales with memory allocation.
- **Concurrency limits** — how many parallel invocations are allowed; important to prevent a Lambda from overwhelming a downstream RDS instance.

> **Why it matters:** Choosing EC2 vs ECS/Fargate vs Lambda comes down to: how long does the workload run, is it request/response or event-driven, and how much infrastructure do you want to own. This is a common system-design interview question.

2. Storage

2.1 S3 (Simple Storage Service)

Object storage — think "a filing cabinet for files," not a mounted disk. You upload/download objects using the AWS SDK, not a file path.

- **Buckets & Keys** — a bucket is the container, a key is the "path" identifying an object inside it.
- **Storage classes** — Standard (frequent access), Infrequent Access (cheaper, retrieval cost), Glacier (archival, slow retrieval, very cheap).
- **Versioning** — keeps historical versions of an object when overwritten.
- **Lifecycle policies** — automatically transition or delete objects after N days (e.g., move logs to Glacier after 90 days).
- **Presigned URLs** — a temporary, time-limited URL you generate server-side so a client can upload/download directly to/from S3 without going through your app.
- **Encryption** — SSE-S3 (AWS-managed keys) or SSE-KMS (customer-managed keys via KMS).

> **Java usage:** `S3Client` from the AWS SDK v2 — `putObject()`, `getObject()`, `generatePresignedUrl()`. Common for storing uploaded files, reports, and backups.

2.2 EBS (Elastic Block Store)

A virtual hard disk attached to exactly one EC2 instance at a time — used for the instance's root volume or additional local storage.

- **Volume types** — `gp3` (general purpose SSD, most common), `io2` (high-performance, for databases), `st1` (throughput-optimized HDD, for large sequential workloads).
- **Snapshots** — point-in-time backups of a volume, stored in S3 behind the scenes.

2.3 EFS (Elastic File System)

A shared network filesystem multiple EC2 instances or containers can mount simultaneously — like an NFS share. Useful only when several compute nodes genuinely need to read/write the same files.

3. Databases

3.1 RDS (Relational Database Service)

A managed relational database — MySQL, PostgreSQL, Oracle, SQL Server, MariaDB. Your JDBC code doesn't change; AWS just handles patching, backups, and failover.

- **Multi-AZ** — a standby replica in a different Availability Zone for automatic failover during outages.
- **Read Replicas** — read-only copies to offload read traffic from the primary.
- **Parameter Groups / Option Groups** — database engine configuration settings managed as a reusable set.
- **Automated backups & snapshots** — point-in-time recovery windows.
- **RDS Proxy** — a connection pooler sitting in front of RDS, critical when Lambda functions open too many short-lived DB connections and exhaust the database's connection limit.

3.2 Aurora

AWS's own MySQL/PostgreSQL-compatible engine — same JDBC driver, but faster, more scalable, and with faster replica lag than standard RDS. Aurora Serverless scales capacity automatically based on load.

3.3 DynamoDB

A NoSQL key-value/document store. A fundamentally different mental model than JPA/Hibernate: no joins, no ad-hoc queries — you design your table schema around known access patterns up front.

- **Partition key (+ optional sort key)** — determines how data is distributed and how you query it.
- **GSI / LSI (Global/Local Secondary Indexes)** — alternate ways to query the same table beyond the primary key.
- **Capacity modes** — On-Demand (pay per request, scales automatically) vs Provisioned (fixed read/write capacity, cheaper at predictable scale).
- **DynamoDB Streams** — a change log of table modifications, often used to trigger a Lambda function on insert/update/delete.

> **Java usage:** accessed via the AWS SDK's `DynamoDbClient` or the higher-level Enhanced Client (annotation-based mapping similar to JPA entities, but not the same query semantics).

3.4 ElastiCache

Managed in-memory caching — Redis or Memcached — used exactly as you'd use a self-hosted Redis instance, just without patching/failover management.

- **Redis** — supports persistence, replication, pub/sub, richer data structures. Most common choice.

- **Memcached** — simpler, multi-threaded, purely in-memory with no persistence — used for straightforward caching only.
- Typical uses: session storage, caching expensive query results, rate-limiting counters.

Need	Reach for
Transactional data, complex relationships, existing JPA models	RDS / Aurora
Massive scale, simple access patterns, single-digit-ms latency	DynamoDB
Speeding up repeated reads, session state	ElastiCache (Redis)

4. Networking

4.1 VPC (Virtual Private Cloud)

Your own isolated network inside AWS. Every resource you launch lives inside a VPC.

- **CIDR block** — the IP address range assigned to the VPC (e.g., `10.0.0.0/16`).
- **Subnets** — smaller IP ranges carved out of the VPC, each tied to one Availability Zone.
  - **Public subnet** — has a route to an Internet Gateway; resources here can be reached from/reach the internet directly.
  - **Private subnet** — no direct internet route; your app servers and databases usually live here.
- **Internet Gateway (IGW)** — allows public subnet resources to communicate with the internet.
- **NAT Gateway** — lives in a public subnet and lets private subnet resources initiate outbound internet connections (e.g., calling an external API) without being reachable from outside.
- **Route Tables** — rules that determine where network traffic from a subnet is directed.

4.2 Security Groups vs Network ACLs

- **Security Groups** — stateful, instance-level firewall. "Allow inbound port 8080 from this load balancer's security group only." This is the #1 thing you'll debug when a connection mysteriously fails.
- **Network ACLs** — stateless, subnet-level firewall. Less commonly touched day-to-day, but useful to know when a whole subnet needs a blanket rule.

4.3 Load Balancers

- **ALB (Application Load Balancer)** — operates at the HTTP layer; supports path-based and host-based routing (e.g., `/api/orders` to one target group, `/api/users` to another). The standard choice in front of a Spring Boot service.
- **NLB (Network Load Balancer)** — operates at the TCP layer; ultra-low latency, very high throughput. Used for non-HTTP protocols or extreme performance needs.
- **Target Groups** — the set of instances/containers/IPs the load balancer forwards traffic to, along with health check configuration.
- **Health checks** — the load balancer pings an endpoint (e.g., `/actuator/health`) to decide whether to keep sending traffic to an instance.

4.4 Route 53

AWS's DNS service.

- **Hosted Zone** — a container for DNS records for a domain.
- **Record types** — A, CNAME, Alias (AWS-specific, points directly to an AWS resource like a load balancer without an extra lookup).
- **Routing policies** — simple, weighted (split traffic by percentage), latency-based, failover.

4.5 CloudFront

AWS's CDN — caches content (static assets, sometimes API responses) at edge locations close to users, reducing latency and load on your origin servers.

5. Messaging & Integration

Highly relevant once you move from a monolith to microservices — this is how Java services talk to each other without a direct synchronous REST call.

5.1 SQS (Simple Queue Service)

- A producer pushes a message; a consumer polls and processes it. Decouples services — e.g., an order-service pushes an event, an inventory-service consumes it independently.
- **Standard queue** — near-unlimited throughput, at-least-once delivery, no strict ordering.
- **FIFO queue** — strict ordering and exactly-once processing, lower throughput.
- **Visibility timeout** — the window during which a message is hidden from other consumers after being read, so it isn't processed twice while being handled.
- **Dead-Letter Queue (DLQ)** — where messages go after repeated processing failures, so you can inspect/retry them separately.
- **Java usage:** raw `SqsClient`, or Spring Cloud AWS's `@SqsListener` annotation for a much more Spring-idiomatic consumer.

5.2 SNS (Simple Notification Service)

- Publish/subscribe messaging — one published message fans out to multiple subscribers at once (email, SQS queues, Lambda functions, HTTP endpoints).
- Common pattern: SNS topic fans out to several SQS queues, one per interested microservice.

5.3 EventBridge

- An event bus with rule-based routing — events matching a pattern get routed to specific targets. More structured than SNS for building genuine event-driven architectures across many services.

5.4 Kinesis

- Real-time streaming data, conceptually similar to Kafka. Used for high-throughput event streams — clickstream data, log aggregation, real-time analytics pipelines.
- **Shards** — the unit of throughput/parallelism within a stream.

5.5 Step Functions

- Orchestrates multi-step workflows (often chains of Lambda functions) as a visual state machine — useful for long-running business processes with retries, branching, and error handling built in.

6. Security & Identity

6.1 IAM (Identity and Access Management)

- **Users** — individual identities (people), typically with MFA and console/API access.
- **Groups** — collections of users sharing the same permissions.
- **Roles** — identities assumed by AWS resources (EC2, Lambda, ECS tasks) rather than people — this is how your Java app gets AWS permissions *without hardcoded credentials*.
- **Policies** — JSON documents defining exactly what actions are allowed/denied on which resources.
- **Least privilege** — the guiding principle: grant only the specific permissions a service needs, nothing broader.

> **Rule of thumb:** never hardcode AWS access keys in your Java code or config files. Attach an IAM role to the EC2 instance/ECS task/Lambda function, and the AWS SDK's default credential provider chain picks it up automatically.

6.2 KMS (Key Management Service)

Manages encryption keys used to encrypt data at rest — RDS storage, S3 objects, EBS volumes, Secrets Manager entries. You reference a KMS key; AWS handles the actual cryptographic operations.

6.3 Secrets Manager vs Parameter Store

- **Secrets Manager** — stores secrets (DB passwords, API keys) with automatic rotation support. Slight per-secret cost.
- **Systems Manager Parameter Store** — stores config values and secrets (with encryption via KMS); free tier is generous, but rotation is manual.
- Your Spring Boot app fetches these at startup instead of reading plaintext values from `application.properties`.

6.4 Cognito

Managed user authentication.

- **User Pools** — user directory handling sign-up/sign-in, issuing JWT tokens.
- **Identity Pools** — grant temporary AWS credentials to authenticated (or guest) users, e.g., to let a mobile app upload directly to S3.
- An alternative to rolling your own Spring Security + user table + password reset flows.

7. API Layer

7.1 API Gateway

Sits in front of your backend (commonly Lambda, but can also front EC2/ECS) to expose REST/HTTP APIs with built-in cross-cutting concerns.

- **REST API vs HTTP API** — REST API has more features (request validation, usage plans); HTTP API is simpler, cheaper, and lower-latency for basic proxying needs.
- **Throttling** — rate-limiting requests per client/API key.
- **Authorizers** — Lambda-based or Cognito-based checks run before a request reaches your backend.
- **Usage plans & API keys** — managing external/partner access with quotas.

> **When you actually need it:** if you're running Spring Boot on ECS/EC2 behind an ALB, you often skip API Gateway entirely — it earns its place mainly in front of Lambda, or when you specifically need built-in throttling/API-key management for external consumers.

8. CI/CD & Infrastructure as Code

8.1 CodePipeline / CodeBuild / CodeDeploy

- **CodePipeline** — orchestrates the overall release workflow (source → build → test → deploy stages).
- **CodeBuild** — runs your Maven/Gradle build and tests in a managed environment.
- **CodeDeploy** — automates deployment to EC2/ECS/Lambda, supporting blue/green and rolling deployments.
- Many organizations use Jenkins or GitHub Actions for the pipeline itself and only use AWS native tools as the deployment target — both patterns are common.

8.2 CloudFormation

Defines infrastructure (VPCs, RDS instances, ECS services, IAM roles) as declarative YAML/JSON templates — "infrastructure as code." AWS provisions and updates resources to match the template.

8.3 AWS CDK (Cloud Development Kit)

Same goal as CloudFormation, but you define infrastructure using actual code — including Java. Worth knowing specifically because, as a Java developer, CDK for Java lets you express infra using familiar classes and loops instead of hand-writing YAML.

9. Observability

9.1 CloudWatch

- **Logs** — your Spring Boot app's stdout/stderr is captured here automatically on ECS/Lambda; you query it with CloudWatch Logs Insights.
- **Metrics** — CPU, memory, request counts, custom application metrics (e.g., via Micrometer).
- **Alarms** — trigger notifications (via SNS) or auto-scaling actions when a metric crosses a threshold — e.g., alert when error rate or SQS queue depth spikes.
- **Dashboards** — visual, at-a-glance views of key metrics.

9.2 X-Ray

Distributed tracing — visualizes a single request's path across multiple microservices or Lambda invocations, which is invaluable for pinpointing which hop in a service chain is causing latency.

9.3 CloudTrail

An audit log of every API call made against your AWS account (who did what, from where). More of an operations/security/compliance concern than day-to-day development, but essential when investigating "who changed this resource."

10. Java-Specific Tooling on AWS

10.1 AWS SDK for Java v2

Package: `software.amazon.awssdk`. The modern SDK (v2, not the legacy v1) — supports both synchronous and asynchronous clients, and is what you use to talk to S3, DynamoDB, SQS, and virtually every other AWS service directly from your code.

10.2 Spring Cloud AWS

Spring-idiomatic wrappers over the raw SDK — dramatically less boilerplate:

- `@SqsListener` for consuming queue messages declaratively.
- An `S3Template` for simplified object operations.
- Auto-configuration that wires up RDS connection details or Secrets Manager values directly into Spring beans.

10.3 Lambda with Java

- Implement `RequestHandler<InputType, OutputType>` as the function entry point.
- Package as a "fat jar" (all dependencies included) or use Lambda Layers to separate dependencies from your code.
- Watch cold start time closely — it's the most common Java-on-Lambda complaint; SnapStart and minimizing dependency footprint both help.

10.4 Local development & testing

- **LocalStack** — runs emulated AWS services locally in Docker, so you can test S3/SQS/DynamoDB integrations without hitting real AWS.
- **Testcontainers** — often paired with LocalStack or real Docker images (Postgres, Redis) for realistic integration tests in your Java test suite.

Suggested learning order for ramping up quickly: EC2/ECS + VPC/Security Groups + RDS + S3 + IAM roles + SQS/SNS first (covers most real-world Java backend work), then Lambda + API Gateway, then CloudWatch/X-Ray for debugging, then CDK/CloudFormation once you start touching infrastructure provisioning yourself.